

Асинхронные запросы в БД

В ряде сценариев возникает необходимость запрашивать независимые данные из нескольких баз, осуществлять независимые от результатов запросов в БД сетевые вызовы или производить не связанные с результатами ресурсоёмкие вычисления или другие длительные операции. Выполнять запросы и независимые операции последовательно не эффективно, так как общая длительность исполнения будет простой суммой времён каждой операции. Тогда как одновременное исполнение запросов с независимыми операциями ограничивается временем наиболее длительной стадии.

Платформа предоставляет механизм асинхронного исполнения запросов в БД, позволяющий организовать выполнение независимых операций во время ожидания данных по запросу.

Отправка асинхронного SQL запроса

Отсылка асинхронных запросов производится аналогично синхронным.

C++:

```
1 auto statement = dbi::GetConnectedDataDB()->CreateStatement();
2 auto async_result = statement->AsyncQuery( L"SELECT что-то FROM откуда-то"_sv );
```

Python:

```
1 statement = sbis.CreateStatement()
2 async_result = statement.AsyncQuery('SELECT что-то FROM откуда-то')
```

Но в отличие от синхронного интерфейса асинхронный возвращает не курсор на выбранные данные, а объект, предоставляющий механизм ожидания и извлечения результатов запроса, экземпляра класса `IAsyncQueryResult` ([C++](#), [Python](#)).

Как и в случае с синхронными запросами, в асинхронный запрос можно помещать несколько SQL инструкций.

C++:

```
1 auto statement = dbi::GetConnectedDataDB()->CreateStatement();
2 auto async_result = statement->AsyncQuery(
3     L"SELECT что-то FROM откуда-то; "
4     L"SELECT что-то-ещё FROM откуда-то-ещё;"_sv );
```

Python:

```
1 statement = sbis.CreateStatement()
2 async_result = statement.AsyncQuery('SELECT что-то FROM откуда-то; '
3                                     'SELECT что-то-ещё FROM откуда-то-ещё;')
```

Однако результаты таких запросов имеют существенное отличие при использовании асинхронного интерфейса: синхронный запрос возвращает результаты только последней SQL инструкции, в то время как асинхронный запрос последовательно выдаёт результаты всех инструкций. Это даёт возможность оптимизировать исполнение запросов, собирая несколько инструкций в один запрос, за счёт сокращения накладных расходов, связанных с передачей запроса на сервер БД и приёма ответа.

Отправка асинхронного подготовленного SQL запроса

C++:

```
1 auto statement = dbi::GetConnectedDataDB()->CreateStatement();
2 auto async_result = statement->AsyncPreparedQuery( L"имя-запроса"_sv,
```

3

откуда-то"_sv);

L"SELECT что-то FROM

Python:

```

1 statement = sbis.CreateStatement()
2 async_result = statement.AsyncPreparedQuery('имя-запроса', 'SELECT что-то FROM откуда-то')
```

Проверка готовности результата

Объект асинхронных результатов имеет индикатор готовности очередного результата.

C++:

```

1 if( async_result->IsReady() )
2     ...
```

Python:

```

1 if async_result.IsReady():
2     ...
```

Ожидание готовности результата

Интерфейс асинхронных запросов реализует функциональность для ожидания как одного, так и нескольких запросов, в том числе с ограничением по времени.

C++:

```

1 // дождаться готовности результата без ограничения по времени
2 async_result->Wait();
3 // ожидать готовности результата в течение указанного периода времени
4 // вернёт false, если истекло время ожидания
5 if( async_result->WaitFor( Milliseconds{ 100 } ) )
6     ...
7 // ожидать готовности результата до указанного момента времени
8 // вернёт false, если истекло время ожидания
9 if( async_result->WaitUntil( DateTime::Now() + Milliseconds{ 100 } ) )
10    ...
11
12 std::unique_ptr< IAsyncQueryResult > async_results[ 5 ];
13 auto adaptor = []( std::unique_ptr< IAsyncQueryResult >& result )
14     -> Waitable& { return *result; };
15
16 // дождаться готовности результата всех запросов без ограничения по времени
17 WaitForAll( std::begin( async_results ), std::end( async_results ),
18     adaptor );
19 // ожидать готовности результата всех запросов в течение указанного периода времени
20 // вернёт false, если истекло время ожидания
21 if( WaitForAllFor( std::begin( async_results ), std::end( async_results ),
22     Milliseconds{ 100 }, adaptor ) )
23     ...
```

```

23
24 // дождаться готовности результата любого из запросов без ограничения
    по времени
25 auto iterator = WaitForAny( std::begin( async_results ), std::end(
    async_results ),
26                             adaptor );
27 // ожидать готовности результата любого из запросов в течение
    указанного периода времени
28 // вернёт false, если истекло время ожидания
29 if( auto optional_iterator =
30     WaitForAnyFor( std::begin( async_results ), std::end(
    async_results ),
31                   Milliseconds{ 100 }, adaptor ) )
32     ...

```

Python:

```

1 # дождаться готовности результата всех запросов
2 sbis.WaitForAllQueries([async_result_1, async_result_2])
3
4 # дождаться готовности результата любого из запросов
5 # вернёт курсор на данные готово результата
6 cursor = sbis.WaitForAnyQuery([async_result_1, async_result_2])

```

Извлечение результата

Очередной результат запроса можно получить, вызвав метод `Next` у `IAsyncQueryResult` ([C++](#), [Python](#)). При неготовности результата вызов заблокируется, пока данные не будут получены с сервера БД.

C++:

```
auto cursor = async_result->Next();
```

Python:

```
cursor = async_result.Next()
```

Подробнее о работе с курсором, полученным в качестве результата запроса, читайте [здесь](#).

```

1 # перебираем результаты
2 cursor = async_result.Next()
3 while cursor:
4
5     # перебираем записи в составе одного результата
6     while cursor.Next():
7         rec = cursor.Data()
8         DoSomethingWithRec( rec )
9     cursor = async_result.Next()

```

Отмена запроса

Механизм асинхронных операций позволяет отменить исполняемый запрос, отбросив все невыбранные и неполученные результаты. Следует учитывать, что прерывание не гарантируется, так как к моменту получения сервером БД команды обработка запроса может уже быть завершена.

C++:

```
async_result->Cancel();
```

Python:

```
async_result.Cancel()
```

Особенности работы с асинхронными запросами

Интерфейс асинхронных запросов поддерживает только один активный запрос и на всё время исполнения запроса блокирует соединение. Поэтому попытка выполнить другой запрос, хоть асинхронный, хоть синхронный, до завершения текущего может привести к взаимной блокировке.

По той же причине не следует сохранять объект асинхронных результатов от соединений менеджера БД (соединений, полученных через семейство функций [GetConnectedDataDB](#), [GetDataDB](#), [GetConnectedDatabase](#), [CreateStatement](#)) за пределами контекста исполнения метода БЛ. Иначе потенциально можно заблокировать сторонний метод, если он получит то же соединение от менеджера БД.

`IAsyncQueryResult` является RAII-объектом и по освобождению прерывает незавершённый запрос, отбрасывая невыбранные данные. При этом логика определения завершённости запроса опирается на выборку нулевого результата из соединения, так как платформа не располагает информацией о количестве инструкций в SQL запросе. Поэтому, дабы избежать накладных расходов прерывания, следует выбирать все результаты завершённого асинхронного запроса.

В силу особенностей управления временем жизни объектов в Python при работе с асинхронными запросами необходимо оборачивать `IAsyncQueryResult` в конструкцию `with`:

```
1 with statement.AsyncQuery('SELECT что-то FROM откуда-то') as  
  async_result:  
2     cursor = async_result.Next()
```



Поддержка

В Платформе версии 21.2000 асинхронные вызовы поддерживаются только драйвером БД PostgreSQL.